

AR Project Mini Report

Concept

My project is a small AR Christmas ornament experience that lives on the real Christmas tree at the university. Instead of augmenting the whole tree, I focus on a single voxel-style bauble that appears in AR when you scan a printed marker attached to the tree.

When the user points their phone at the Christmas tree marker, a voxel bauble appears anchored in 3D space around the marker. It is a classic spherical ornament but built from voxels – blocky 3D pixels. The user can:

1. Slide through different voxel "resolutions" from an extremely chunky 1-voxel version up to an 11-voxel diameter sphere.
2. Press a "Lights" button to toggle a warm glowing version on and off.

The aesthetic is personal and nostalgic. I grew up getting video games like Pokémon and old Mario titles at Christmas, and their low-res, pixelated look influences this design. The ornament is intentionally simple: a red, shiny bauble that reads clearly even at low resolutions. At the lowest level it looks like a single red cube; at higher levels it approaches a sphere while keeping the blocky flavour.

There is also an "explainer" side: by sliding through voxel diameters, the user sees how resolution affects shape. With more voxels across the sphere, it becomes more recognisable and smoother while staying discretised into cubes. The piece is both playful and a visualisation of discretisation and resolution.

Target users are students, staff and visitors walking past the tree. The moment is casual: they scan the marker, see the bauble appear, then play with the slider and light toggle. Ideally it sparks quick comments like "It's so chunky at 1 voxel" or "This feels like Minecraft/Pokémon". Even if they only interact for a minute, I want them to feel a small hit of nostalgia and curiosity.

Research Snapshot

Formal research for this project was limited. I did not do a structured literature review or a systematic survey of AR Christmas apps. Instead, I relied on basic knowledge of AR image tracking, Unity docs, and general exposure to games and AR examples.

In AR Foundation, image tracking uses an `XReferenceImageLibrary` containing one or more reference images with real-world physical sizes. At runtime, `ARTrackedImageManager` and the underlying ARCore/ARKit SDKs detect these images in the camera feed. When an image is recognised, AR Foundation creates an `ARTrackedImage` with position, rotation and

size, and I instantiate a prefab at that pose so the virtual object appears stuck to the real marker. Correct physical size is important so the bauble appears at a believable scale relative to the print.

For marker design, I roughly followed standard guidance: high contrast, lots of visual detail, distinct features and edges, and minimal repetitive patterns. My marker is a printed Christmas tree image attached to the real tree. It is festive but not technically optimal: lots of green area, some repeated textures, and visual “noise” around it from the real tree. Its roughly A4 size helps visibility, but a high-contrast abstract marker would likely track more robustly.

I did not deeply research precedents for AR Christmas experiences or voxel visualisations. Inspirations were mostly personal: classic pixel-art games and modern voxel art like Minecraft. The core idea was: “What if a Christmas bauble looked like it had been rendered at different resolutions in an old game engine, and you could scrub through those levels in AR?”

Looking back, there are clear gaps. With more time I would have:

- Read Unity’s AR tracking and marker best-practice docs more thoroughly.
- Studied other AR Christmas/holiday installations for interaction patterns and onboarding.
- Consulted AR UX guidelines to make scanning and interaction self-explanatory.

Tooling and testing also limited research depth. Developing on Windows for mobile AR (including iOS) meant a lot of effort went into getting builds and troubleshooting instead of exploring literature. iOS testing usually requires a Mac and Xcode; without that, I relied on the Unity Editor, which pushed me towards “just build and see” rather than deeper research.

Even so, the project is grounded in core AR concepts: reference image libraries, physical marker size, and marker trackability. The rest of the inspiration is mostly personal gaming history and general awareness of voxel aesthetics rather than academic sources.

Design Summary

From the user’s perspective, the flow is simple. On launch, the app goes straight to the AR camera view. The expectation is that they are near the university Christmas tree or have the printed marker. They point the device at the marker; once recognised, the voxel bauble prefab spawns and is anchored to that pose, appearing to float slightly in front of the printed image.

On screen, a Lights button and a slider sit near the bottom, out of the way of the main AR content but reachable with a thumb. Once the ornament appears, the user presses the Lights button to toggle the internal glow and uses the slider to change voxel resolution.

Onboarding is minimal: there is no dedicated overlay instructing users to “Scan the Christmas tree marker” or how to use the controls. In a classroom or guided demo, verbal instructions

compensate, but in a public setting this is a UX weakness. A simple text like “Point your camera at the Christmas tree marker, then use the slider and Lights button” would improve clarity.

The main interactions are:

1. **Voxel resolution slider**

The slider switches between discrete voxel models with diameters 1, 3, 7, 9 and 11. The bauble maintains roughly constant world-space size; what changes is voxel count and size. At diameter 1 it is a single cube; at 3 it becomes a small chunky cluster; by 9 and 11 it looks relatively smooth but still built from cubes. I originally planned a 1, 3, 5, 7, 9, 11 progression, but the 5-voxel version did not make it into the final build, leaving a noticeable jump from 3 to 7.

2. **Lights toggle button**

The Lights button switches between a no-light and with-light version of the currently active voxel model. The lit version has an internal warm point light, like a candle or LED inside a translucent red shell. This emphasises a cozy Christmas feeling.

There is no audio in this version, which keeps the experience visually focused but also makes it feel a bit quiet and less festive. Adding even a soft chime for button presses would help.

Visually, the design is constrained and iconic: a red, moderately shiny bauble. Red reads well against many backgrounds (especially tree greens) and remains recognisable as an ornament even at low resolution. The main “hook” is the change in voxel resolution, not detailed textures. At diameter 1 you still understand it as “the bauble”, just extremely simplified; at 11 it is smoother yet still clearly digital and blocky.

With more time, I would add clearer on-screen guidance and a small label near the slider such as “Voxel Diameter: 3” to highlight the educational angle. I would also refine UI visuals—button icons, click feedback, and a more distinct slider thumb—to move beyond the default Unity look.

Implementation & Performance

The project uses Unity 2022.3.1f1 (LTS) with AR Foundation and the ARCore/ARKit plugins. The AR setup follows the standard pattern: `ARSession`, `ARSessionOrigin`, an AR Camera, and an `ARTrackedImageManager`. The manager references an `XRReferenceImageLibrary` containing the printed Christmas tree marker with an approximate real-world A4-sized width so pose and scale estimations are reasonable.

When the marker is detected, `ARTrackedImageManager` instantiates a root ornament prefab anchored to the tracked image. Under this root is a hierarchy of child objects for each voxel resolution and its light/no-light variants, e.g.:

- `VoxelLevel_1`
 - `VoxelLevel_1_NoLight`
 - `VoxelLevel_1_WithLight`
- `VoxelLevel_3` ... etc. up to 11

Each “NoLight” child holds the voxel geometry with a red material; each “WithLight” child is the same geometry plus a warm point light near the centre.

Each voxel ornament model was created separately for its diameter. The overall radius stays roughly constant in world units, so as voxel count increases, individual cubes become smaller and more numerous, but the bauble occupies about the same AR volume. This avoids jarring scale changes when the user moves the slider.

The UI lives on a Canvas set to Screen Space – Camera with the AR Camera as the render camera. The Canvas holds the Lights button and slider. A controller script on the spawned root tracks the current voxel level and light state.

The slider is wired to a method that maps its value to one of the five discrete levels. When the value enters a new range, the script deactivates all voxel parents and activates the matching level, respecting the current light state (`_WithLight` vs `_NoLight`).

The Lights button’s OnClick calls another method on the same controller, which flips a `lightsOn` boolean and updates the active children for the current level.

Platform testing was a major constraint. The assignment targets mobile AR on iOS and Android, but I primarily developed on Windows. Proper iOS testing requires a Mac and Xcode; without a smooth pipeline, most iteration happened in the Unity Editor. That was enough to test logic (slider changes, button toggles) but not full on-device behaviour.

This led to issues such as UI buttons not always responding to touch on the iPhone build, despite working in the Editor, and the AR content sometimes failing to appear when scanning the marker on iOS. Without reliable Xcode logs or live debugging, I could only guess at the root causes (reference image setup, prefab assignment, build settings). Time limitations meant these platform-specific problems remained partly unresolved.

Performance-wise, the scene is very light: simple cube or low-poly meshes, a basic mobile-friendly material, no heavy shaders or post-processing, and only one voxel level active at a time with a single internal light. On a mid-range mobile device, it should run smoothly, but testing under real-world conditions around the campus tree was limited.

Evaluation & Reflection

Formal evaluation was heavily constrained by hardware and build issues. I mainly tested in the Unity Editor and was not able to run stable builds on devices: I lacked consistent access to a Mac for iPhone builds and could not complete Android deployment. As a result, the current AR build does not reliably show the model on the marker on device, and I could not conduct a proper user study.

In the Editor, the flow behaves as intended: when the simulated camera detects the marker, the prefab spawns at a sensible position and scale and appears anchored. The UI works: the Lights button toggles between tree variants and state changes are clearly visible. Conceptually, the lit vs unlit versions communicate a simple, playful ornament, and the one-button interaction is easy to understand.

But the lack of on-device validation means the core AR feature—having the tree appear on the physical printed marker—remains unproven. In the current iOS build, the model does not appear, and without iterative device testing I could only hypothesise about configuration errors instead of diagnosing them early.

Despite this, the concept feels promising: a small AR scene where a physical Christmas marker “summons” a digital tree or ornament that changes mood via a single button. Given more time and working builds, I would want to see how users discover the button, how often they toggle lights, and whether it becomes a playful on/off ritual around the real tree.

If I could continue, my priorities would be:

- Fixing the marker-tracking issue so the model consistently appears on the real marker.
- Making the visual difference between lit and unlit states stronger across lighting conditions.
- Adding minimal onboarding text or graphics so scanning and interaction are obvious.
- Iterating on marker artwork and size for better tracking robustness.
- Running at least a small in-situ user test around an actual Christmas tree.

Reflecting on the process, the biggest blocker was the iOS build and testing pipeline. Developing AR for iPhone from Windows, without reliable Mac/Xcode access, made rapid iteration nearly impossible and turned debugging into guesswork. In future projects, securing stable access to a Mac and an AR-capable test device would be a critical early milestone so I can catch configuration errors sooner and invest more energy into design and user experience rather than pipeline issues.

Even with these constraints, the project was a valuable learning experience. I implemented a prefab with multiple visual states controlled by UI, strengthened my understanding of AR Foundation image tracking and prefab instantiation via `ARTrackedImageManager`, and appreciated the importance of correctly wiring UI to runtime-spawned objects. Above all, it underlined how essential real-device testing is for AR: seemingly simple goals like “make the model appear on the marker” can fail silently without it.

